



▶ Watch now

Your skills are leaving your hands. Don't let a rent-a-brain keep them.

Open Brain kept your memory yours. Open Skills keeps the way you work yours, not rented back by whichever AI app wins this month.



NATE

JUN 19, 2026 PAID

You finally got an agent working exactly how you wanted, and then you switched tools and it broke. That small, maddening moment is becoming one of the most expensive problems in knowledge work, and it is the second act of a problem that began with memory.

Open Brain was about memory leaving our heads. Notes, transcripts, Slack threads, calendars, meeting summaries, project histories. Huge

parts of what we know had already moved into software. The trouble was that AI could not reliably use that memory, and when it could, the memory was trapped inside one company's product. Open Brain made one claim: your memory should be yours. Whoever owns the memory owns the starting point, and the architecture itself was never the interesting part. If your context lives in one app, every new tool makes you start over and every better model arrives with a switching cost.

Open Skills is the same fight, one level up, and more urgent. Our skills are leaving our hands now too, and that should feel uncomfortable. For most of human history a skill lived in a person. You knew how to research, write, code, test, review work, and recover when something broke. You carried the standards, the shortcuts, the taste, and the checks that kept bad work from becoming real damage. AI is turning that into software. Your way of working can now become prompts, SKILL.md files, runbooks, scripts, MCP configs, permission boundaries, and agent workflows. The thing that used to live in your hands can now live in a harness.

That is one of the biggest opportunities in AI. It is also one of the biggest ownership fights. Because if your skills are going to live outside your hands, they should not belong to a rent-a-brain. Not to Claude because it shipped the best skill format this quarter, not to Codex because it gave you the best workbench, not to Cursor because the project started there, not to ChatGPT because the subscription was convenient. They should be yours. Visible, movable, inspectable, testable, and available wherever you work.

Today almost none of that is true. The prompt copies over. The intention copies over. The skill does not. So you rebuild it from memory, a teammate improves a different copy and never tells you, and your best workflow ends up stranded in a chat history nobody can find again. You are not short on intelligence or context. You are short on a way to carry the procedure itself.

That gap has a cost, and it compounds. Every tool switch becomes a rebuild. Every new hire starts from scratch. Every improvement risks dying as one person's private habit. This is not a hobby problem. If agents are part of how you do your job, the skills you build with them are career capital, and right now most of that capital is rented back to you by whichever vendor's tool you happened to build it in. Putting it back in your hands is one of the real practical projects of 2026, and it is bigger than any single file format.

Here's what's inside:

- **Open Skills, the library.** A public set of agent skills and runbooks you can install today, built on one rule: the way you work should be yours, not rented back to you.
- **The debt you didn't know you were carrying.** The four ways this breakage shows up at work, and the name for what you keep paying down.
- **Prompt, memory, skill.** Three things people collapse into one, and why only one of them survives a model change.
- **The work package.** A checklist that separates a skill you actually own from a lucky setup you happened to land in one app.
- **One messy workflow, every tool.** A real support-billing process rebuilt so it travels across Codex, Claude Code, and Cursor instead of dying on contact.
- **The one-question test.** The single thing to ask about any workflow to find out whether you own it or rent it.

The pain is already here, but so is the fix, and it is closer than it looks. You do not need a new platform or a new subscription. You need to own the way you already work and make it move with you. The rest of this is how: start with the workflow that keeps breaking, end with skills that travel anywhere you do.

Subscribers get the full deep-dive and guide,
plus membership to my Slack community!

✓ Subscribed

[LINK: Join the Slack →]

The Slack community is live! It's where I'll be sharing things between articles, where you can get help on builds in real time, and where the fastest conversations in this community are already happening. I'll see you in there!

[LINK: Grab the guide]

Before we get into it, I do not want to bury the most useful part. I took the way I actually work with agents and pulled it into a public library, so you do not have to rebuild any of this from scratch. It is called Open Skills: 31 agent skills across 7 categories, plus 7 runbooks that show how those skills combine into real workflows. Each one is a setup prompt you can copy, install in your own environment, and adapt to your own work. The point is not to hand you a pile of clever prompts. It is to give you working knowledge you can carry between tools instead of renting it back from whichever app you happened to build it in. Grab the guide here.

The skill that worked yesterday and broke today

Start with one that keeps breaking. A developer gets a skill working, and not in a toy way. It actually works. It knows the repo conventions. It knows when to use live search. It knows which files are safe to touch. It knows which test matters. It knows when to stop and ask. It can run a

script. It can call the right tool. It leaves a useful handoff instead of a confident shrug.

Then the developer tries to use the same skill somewhere else.

Maybe the original was in Claude Code and the team uses Codex at work. Maybe the project moved from Cursor rules to an agent skills folder. Maybe half the work happens in one tool and half in another. Maybe the user has a personal skill in `~/claude`, a project skill in a repo, a Cursor rule in `.cursor/rules`, a Codex skill in `~/codex/skills`, and a teammate with a better version in a different branch.

So they copy the markdown file.

And the work falls apart.

The file is in the wrong folder. The frontmatter does not match the target tool's trigger rules. The source skill used lazy loading, but the new tool loads rules only at chat start. The old setup had an MCP server, but the new one does not. The original skill depended on a script, but the new tool treats the file as plain text. The old agent had browser access. The new one does not. The original workflow used hooks or subagents. The new environment has neither. The teammate updated the good version two weeks ago, but nobody copied it over, so now three people are using three different versions of the same supposedly shared skill.

This is the real pain.

Not “skills are confusing” in some abstract product-management sense. Real people are already losing hours to this. They are trying to make AI useful at work, and every time they switch tools, join a company setup, move between personal and work machines, or share a workflow with a teammate, the thing that worked stops working.

That developer is not hitting a one-off bug. They are paying down procedural debt: the working knowledge people build with agents keeps piling up in forms no agent can run again. You can see it in the bloated prompt that makes the agent worse, in the voice and testing standards re-explained from scratch every time you switch tools, in the rules that drift across Cursor and Claude Code and Codex until no two of your agents behave alike. But the version that hurts most is the one people miss. The agent reports done, and the source is stale, the page was never opened on mobile, the change was never tested. Automation did not delete that work. It relocated it downstream into review, where a human still has to catch all of it. That is the cost nobody priced in: the agent did not remove the work, it moved it onto you.

That is the problem Open Skills is meant to solve.

If AI agents are going to become part of how people write, code, research, support customers, publish, analyze, design, test, and operate, then your ways of working cannot be trapped inside one tool's folder structure, one hidden system prompt, one chat history, or one company's preferred harness.

This is going to matter more, not less.

Agent skills are becoming a job skill, and not because every worker needs to love YAML or memorize folder paths. The advantage goes to the person who can turn repeated work into a reliable skill that a teammate can inspect, adapt, and run in another environment, over the person sitting on a private pile of prompts.

The future job skill is not "I know the perfect prompt."

It is "I can make the work travel."

The open standard helps. It does not finish the job.

The obvious answer is a standard format.

That is why SKILL.md matters. It gives the skill a readable home. It lets the author define the name, description, trigger, instructions, references, scripts, examples, and supporting files. A human can inspect it. A repo can version it. Another agent can load it when the situation calls for it.

That is real progress.

But those stories make the limitation obvious: a file format is not the same thing as a working skill.

Copying a SKILL.md from one place to another does not guarantee that the target tool will discover it, load it at the right time, honor the same trigger logic, expose the same tools, use the same permissions, run the same scripts, preserve the same memory boundary, or produce the same proof.

That is why “just port the markdown” fails.

The differences start with loading. One tool keeps rules always on, while the next only reaches for a skill when the task semantically matches. Project instructions get read once at chat start in one place and can be loaded mid-run in another. From there it gets worse. A script that runs bundled with the skill becomes inert text the moment you move it. Hooks the model cannot ignore turn into prose it may compress, dilute, or skip. Subagents with their own context simply do not exist in the next tool over. Browser access comes and goes. The MCP server your skill depends on either connects or asks for a config file the new tool has never heard of.

These differences are not edge cases.

They are the job.

If you are a normal person trying to make AI useful, you do not experience this as “semantic loading variance.” You experience it as: I had this working yesterday, and now it is broken.

You update a skill in one place and forget to copy it to the others. Weeks later you discover you have been using the worse version. A teammate improves a project rule and nobody else sees it. A company wants a shared agent workflow, but half the team is in Claude Code, two people are in Cursor, one person is in Codex, and the official work environment is locked down. Someone pastes the “same” skill into the new tool and gets a different agent.

After enough of that, people stop improving the skills.

That is the killer.

The problem is not only that the tools are fragmented. The problem is that fragmentation punishes the exact behavior we should want. A person learns how to make an agent better. They capture the lesson. They try to share it. Then the system makes that capture hard to move, hard to test, hard to keep in sync, and easy to rot.

So the lesson stays in the person’s head.

That is how the AI future stays weirdly manual.

A skill is working knowledge, not a better prompt

Most people start by saving prompts because prompts are the first reusable thing they can see.

The agent misses something, so you add a rule. It sounds generic, so you add voice guidance. It uses stale facts, so you add a warning. It skips a test, so you add a checklist. It touches the wrong file, so you

add a boundary. It says “done” too early, so you add a definition of done.

At first, this feels like improvement.

Then the prompt becomes a pile.

Personal style, project rules, safety notes, source hierarchy, old failure scars, testing expectations, tool instructions, and one-time caveats all end up in the same box. The model follows some of it. You stop reading all of it. A teammate copies a version and changes it. A new tool arrives and half the guidance no longer maps cleanly.

The issue is not that prompts are bad. A prompt is fine for a one-time ask.

But a skill is a different object.

A prompt says what you want right now.

Memory says what the agent should remember about you or the work.

A skill says how this kind of work should happen again.

That is why skills are stronger than prompts and stronger than memory for repeated agent work. They carry the working knowledge: when to use them, when not to use them, what sources matter, what tools are needed, what the output should look like, what the agent must not do, and what proof counts before the work can be trusted.

Take “write in my voice.”

As a prompt, it is a wish.

As memory, it is a preference.

As a skill, it is a working method. Read the actual source material. Identify the claim. Preserve the author's judgment. Remove generic AI language. Do not add unsupported claims. Rewrite end to end instead of adding patch notes. Check the draft against examples. Return the artifact, not a list of suggestions.

That is why the word "skill" matters.

It is closer to craft than to a checklist.

The same is true for support triage, source checking, code review, browser QA, transcript processing, customer email routing, finance review, board updates, launch briefs, video production, and every repeated task people are starting to give agents.

The useful asset is the captured working knowledge, not the clever sentence.

The solve has to be punchier than a folder of files

Open Skills is my name for making that working knowledge portable.

The solve is simple to say:

Stop saving prompts as if they are the whole workflow.

Save the skill as a complete work package.

That package needs the readable skill file, but it also needs the things that make the skill run in the real world:

- Where it should be installed.
- How the target tool discovers it.
- When it should trigger.

- Which tools it needs.
- Which permissions it requires.
- Which scripts, MCP servers, browser access, hooks, or subagents it depends on.
- What memory it can read or write.
- Which parts are personal, project-local, team-level, public, or private.
- How it stays in sync across tools and teammates.
- What evidence proves the agent actually did the work.
- Which harnesses it has been tested in.

That is the missing layer.

A SKILL.md file is the source. The work package is the portable thing.

This is why the Open Skills documentation set matters. It is not just a list of neat AI tricks. It is pointing toward a public library of agent skills and runbooks with setup prompts, install guidance, categories, examples, and compatibility thinking. The current set has 31 skills, 7 categories, and 7 runbooks. That product surface matters because people need something they can copy, install, adapt, and test.

But the product is not the real point. The claim underneath it is simpler.

You should own your skills the same way Open Brain argued you should own your memory.

Not own in the abstract. Own in the useful sense.

You can see them, move them, diff them, improve them, and share them. You can keep the private ones private, install a skill in your own environment when it belongs to you, and leave it in the repo when it

belongs to the project. You can tell when one is stale, what it needs, and what breaks when it moves.

That is ownership.

And without that, the next wave of agent work will create a new kind of lock-in.

Not just model lock-in.

Skill lock-in.

Your writing skill lives in one app. Your coding habits live in another. Your customer triage process lives in a chat history. Your browser QA checklist lives in a project rule. Your teammate's better version lives in a repo branch. Your MCP setup lives in a config file nobody else has. Your actual working knowledge is scattered across the products you happened to try first.

In that future, nobody owns their AI leverage. Everyone just becomes the migration layer.

Codex made this visible, but the point is bigger than Codex

Codex changed how I think about this because it changes the unit of work.

In a chat interface, the unit is the message. You ask. The model answers. You correct. It answers again.

In Codex, the unit is closer to a run. A run has a goal, files, tools, permissions, terminal output, artifacts, checks, logs, and review. It can branch into child threads. It can use skills. It can show receipts. It can operate inside a workspace instead of floating in a blank chat box.

That makes delegation feel more real.

But it also makes the portability problem harder to ignore.

If my editorial quality gate only works because I know how to steer Codex, I do not own the skill yet. If my browser QA habit only works because one environment exposes the right browser controls, I do not own the skill yet. If my research flow only works because one thread has the right memory and permissions, I do not own the skill yet.

I may have a good local setup.

That is useful.

But I want the working knowledge to travel.

That means the skill has to describe both the work and the harness it expects. The work says: here is the job, here is the judgment, here is the output, here is what good looks like. The harness says: here are the tools, permissions, memory, scripts, connectors, checks, and receipts needed to run it here.

This is where a lot of AI advice is still too soft.

People talk about agents as if they are universal workers. But serious agent work is full of local dependencies. Folder paths. Permission models. Hidden product instructions. Loading behavior. MCP config. Tool availability. Script execution. Browser state. Team conventions. Approval rules.

If we do not make those dependencies visible, people will keep mistaking a working local demo for a portable skill.

What Open Skills is

Open Skills is a public library and a way of thinking about AI work.

The library gives people concrete skills and runbooks. The idea gives them the ownership model: your ways of working should be portable, inspectable, adaptable, and testable across agent tools.

The current documentation set already has the right ingredients:

- A first-class Open Skills product surface, not a buried guide.
- Setup prompts that help an agent install the skill into the user's actual environment.
- Personal and project-local install paths such as `~/.codex/skills`, `~/.claude/skills`, and `.agents/skills`.
- A clear rule that secrets stay out of the skill file.
- Categories that map to real work: infrastructure, research, writing, publishing, video, testing, and agent operations.
- Runbooks that show how skills combine into outcomes.
- A warning not to turn this into another prompt library.

That last warning is the most important.

The world does not need another shelf of clever prompts. It needs a way for people to preserve and move the working knowledge they are building with agents.

None of this is happening in a vacuum. Coding tools already ship their own rule systems, people already share Cursor rules and tool-specific skills, and there are already sync scripts whose whole job is to stop those files from drifting. That is the right direction, and Open Skills is making a narrower bet inside it. The bet is not that skills exist. It is that a portable procedure can be an operating layer underneath the tools rather than another artifact trapped inside one of them. One markdown source generates the tool-specific copies instead of competing with them, with verification built into the contract and scope deciding whether a procedure belongs to you or to the repo. If that turns into

one more list of clever prompts, it failed. If it holds, it is a layer the multi-tool world does not have yet.

The first Open Skills release should feel practical. It should give people skills they can actually install and run: current-information search, media transcription, heavy file ingestion, personal voice, release briefing, frontend taste, browser automation QA, testing memory, visible delegation, session-to-skill extraction, agentic harness design.

Those categories are not marketing.

They are the work.

Current-information search prevents stale facts from getting published. Personal voice stops “write like me” from being a wish. Browser automation QA stops “looks good” from being treated as evidence. Session-to-skill extraction makes sure a hard-won lesson from one run does not disappear into old chat history. The part people skip is the restraint: at the end of a session it asks whether anything recurring and non-obvious actually happened, and usually the honest answer is no. Saying no most of the time is what keeps it honest. A library that promotes every passing preference into a skill becomes the prompt pile it was supposed to replace.

The runbooks make the value clearer.

“Talk to Published” turns a voice memo into a published page. “Release Day” turns live facts into a sourced briefing. “Video Production Line” turns raw talking-head footage into an edited video with supporting assets. “Ship A Page You Can Trust” connects frontend taste, publishing, browser QA, and testing memory. “The Research Engine” turns messy sources into research where every claim traces back to an artifact and every conclusion has been stress-tested. “Delegate And Verify” turns a fuzzy task into supervised agent work with receipts. “Flywheel” turns completed work back into better skills.

That is the point of the library.

Not “look how many skills exist.”

Here is how work stops resetting.

One workflow, start to finish

Take a simple company workflow: support billing triage.

The company has a real way of doing it, even if nobody has written it down cleanly. The support lead knows which billing system is authoritative. One engineer knows how duplicate accounts show up. The founder wrote a refund rule in Slack. The finance person knows which cases require approval. The team has a few prompts, a few labels, a few old notes, and a lot of habit.

Now they want an agent to help.

In one tool, they write a project instruction. In another, they create a skill. In another, they paste a prompt. In another, they have no setup at all. The agent sometimes checks the right export, sometimes trusts the wrong system, sometimes drafts a customer reply without enough evidence, sometimes asks the human for everything, and sometimes acts more confident than the situation deserves.

The human is still the skill.

Open Skills says: capture the working knowledge.

Make a support billing skill. Give it a trigger. Name the real systems. Define the source order. Say what the agent can read and what it cannot change. Define the evidence packet. Define the customer-facing draft standard. Define when a refund requires approval. Define what proof belongs in the final handoff.

Then make the harness requirements visible. This skill needs read access to these exports. It needs no write access to billing. It needs the company refund policy. It needs a safe place for secrets. It needs a way to cite evidence. It needs compatibility notes for Codex, Claude Code, Cursor, or the internal tool. It needs an owner who updates it when the billing workflow changes.

Now the skill can travel.

Not perfectly or magically, but honestly.

A new teammate can read it. A founder can dispute it. A locked-down work AI can say which required capability is missing. A Codex run can leave a local receipt. A Claude workflow can adapt the same skill. An internal support agent can implement the same checks.

That is the difference between a prompt and owned working knowledge.

Useful artifact: the Open Skill work package

Before you call something an Open Skill, check whether it includes the work and the harness.

OPEN SKILLS / WORK RECEIPT

OWNED WORKING KNOWLEDGE

Visible. Movable. Testable.

OPEN SKILL WORK PACKAGE

Before you call something an Open Skill, check whether it includes the work and the harness.

	PART	QUESTION IT MUST ANSWER
DEFINE THE WORK	● Skill name	What repeatable work does this skill own?
	● Trigger	When should an agent use it?
	● Boundary	What nearby work should not use it?
	● Scope	Is it personal, project-local, team-level, public, or private?
	● Source of truth	Which files, systems, docs, people, dashboards, or live surfaces outrank everything else?
	● Inputs	What does the agent need before starting?
EXPOSE THE HARNESS	● Tools	What files, commands, connectors, browsers, APIs, scripts, hooks, MCP servers, or subagents does it require?
	● Permissions	What may the agent read, edit, run, publish, send, or delete? What requires approval?
	● Secrets	Where do credentials and private values live outside the skill file?
	● Memory	What may the skill read from memory, write back to memory, or keep only inside the run?
PROVE THE RESULT	● Work steps	How should the work actually happen?
	● Output	What artifact, handoff, file, draft, report, patch, or page should exist at the end?
	● Proof	What evidence is required before the agent can say done?
	● Stop conditions	When should the agent stop instead of improvising?
KEEP IT ALIVE	● Runbook role	Which larger workflows use this skill with others?
	● Compatibility	Where has this been tested, and what breaks elsewhere?
	● Sync	How does the skill stay aligned across tools, projects, and teammates?
	● Owner	Who maintains it when the work changes?

If you cannot answer these questions, you may still have a useful instruction.

But you do not yet have an Open Skill. You have a local trick.

If you cannot answer those questions, you may still have a useful instruction.

But you do not yet have an Open Skill.

You have a local trick.

How this differs from ownership, maintenance, and Codex

Every Agent Needs an Owner, coming up this Sunday, is the accountability argument. Once an agent is doing real work, a human has to own the job, permissions, review loop, and retirement rule.

Agent Maintenance is the care-and-feeding argument. Agents drift because models change, tools change, routes move, APIs break, permissions expand, and old instructions become wrong.

The Codex guide is the delegation-surface argument. It shows how to use one environment well: persistent threads, local workspaces, child threads, artifacts, checks, permissions, reviews, and real runs.

Open Skills is the ownership argument for working knowledge.

- Who owns the way this work gets done?
- Where does it live?
- Can it move?
- Can a teammate inspect it?
- Can a new work AI install it?
- Can you tell what breaks when it leaves the tool where it was born?

Those questions are the difference between a personal AI habit and a durable skill.

Put your skills back in your hands

Open Brain put memory back in your hands.

Open Skills should put your AI skills back in your hands.

That matters because agents are moving from novelty to work. The valuable people will not only be the ones who know which model is best this week. They will be the ones who can turn repeated work into skills that other people and other tools can actually use.

This is not a small thing.

Every company is going to accumulate AI working knowledge. Some of it will be explicit. Most of it will be scattered across prompts, chats, tool settings, repo files, extensions, scripts, MCP configs, browser habits, and the memory of the person who made it work once.

If that working knowledge stays trapped, every tool switch becomes a rebuild. Every new employee starts from scratch. Every teammate has a slightly different agent. Every improvement risks becoming a private habit instead of a shared capability.

Open Skills is the opposite bet. The skill is visible, the install path is clear, the permissions are bounded, the proof is required, and the limits are honest. The owner is known.

That is how AI work stops resetting.

So start with the painful workflow.

The one you keep re-explaining.

The one that worked in one tool and broke in another.

The one your teammate cannot run because too much of it lives in your head.

The one that would make you angry to rebuild from scratch next month.

Ask one question:

If I had to move this skill tomorrow, what would break?

The answer is your Open Skills roadmap.

[The full library, categories, and runbooks are here.](#)

Coming up

This piece is one move in a larger build. Memory and skills are yours now. What comes next is the layer that runs them.

Tomorrow, find out: am I AI? I settle it once and for all, and I'll warn you now the answer is somewhat anticlimactic.

This Sunday, the Executive Briefing takes on the question that shows up the moment an agent starts doing real work: who owns it. Every Agent Needs an Owner covers the job, the permissions, the review loop, and when to retire an agent that has gone stale.

Then next week the build moves to the loop itself. Wednesday's deep dive argues that agents are really loop managers, a small supervised fleet rather than one genius you prompt and hope. Friday the Open Orchestrator ships, the part of the Open family that uses your task system to coordinate everything the skills can do, so memory, taste, and delivery finally run on one set of rails. If Open Skills is about owning the procedure, the Orchestrator is about running all of them at once.

Related reading

- [Open Brain: why your AI starts from zero every time you open a new chat](#)
- [The two ways to give AI a brain](#)
- [Codex Guide: your first personal agent delegation loop](#)
- [Claude vs. Codex is about whether you steer or dispatch](#)

- [How I think about MCP](#)

I make this Substack thanks to readers like
you!

✓ Subscribed

